SPM Module 2: Parallel Partitioned Hash Join Pipeline

Teja Karuku
Matriculation: 724955

April 2026

1 Introduction

This report details the design, implementation, and performance evaluation of a parallel Partitioned Hash Join pipeline using native C++ threads. In modern database management systems (DBMS) and distributed data processing frameworks, joining large, unordered datasets is a foundational and computationally expensive operation. The partitioned hash join addresses this by dividing massive relations into smaller, cache-resident chunks, allowing for localized hash probing.

Building upon the bitwise partition mapping optimized in Module 1, this second phase parallelizes the entire end-to-end join process. The primary objective is to compute the total number of matches (accounting for duplicates) between two large relations, R and S, while strictly preserving output correctness against a sequential baseline. This report analyzes the varying degrees of parallelism achievable across the algorithm's distinct phases, differentiating between compute-bound and memory-bound operations.

2 Experimental Setup and Methodology

All benchmarks and verifications were executed on a dedicated compute node (`node09`) within the university cluster to eliminate operating system jitter and interference from other users.

Dataset Characteristics: The input parameters were fixed at $N = 50,000,000$ 64-bit integer records for both relation R and relation S, distributed across $P = 256$ partitions. The data was deterministically generated using the `splitmix64` pseudo-random number generator initialized with a fixed seed (`12345`). This ensures that the computational overhead of generating the dataset remains strictly outside the timed execution windows.

Compilation Environment: The source code was compiled using GCC with the following optimization flags: `-O3 -mavx2 -pthread -std=c++20`. The `-O3` flag ensures maximum loop unrolling and auto-vectorization, while `-pthread` enables the POSIX threading backend required by `std::thread`.

3 Parallelization Strategy and Phase Analysis

The partitioned hash join algorithm is comprised of distinct sequential phases, each exhibiting unique computational requirements and memory access patterns. Our implementation utilizes `std::thread` and adopts a hybrid approach: applying a data-parallel strategy for memory-bound phases and a task-parallel strategy for compute-bound phases.

3.1 Phase 1: Local Histograms (Data-Parallel)

To determine the memory footprint of each partition, the algorithm must scan both relations and count the frequency of each partition mapping. In a naive parallel implementation, multiple threads updating a shared global histogram would require costly `std::atomic` operations or `std::mutex` locks, leading to severe pipeline stalls.

To eliminate this, we implemented a data-parallel strategy. The input arrays (R and S) are divided into equal, contiguous chunks based on the number of spawned threads (T). Each thread computes a localized histogram of partition frequencies strictly for its assigned chunk. By ensuring threads write exclusively to their own private memory arrays, we completely avoid race conditions and cache invalidation protocols.

3.2 Phase 2: Global Prefix Sum (Amdahl's Law)

The local histograms must be converted into starting memory offsets (a prefix sum or exclusive scan) to determine where each thread will write its data during the scatter phase.

Because the number of partitions ($P = 256$) is relatively small, the mathematical operations required for this prefix sum are trivial. In accordance with Amdahl's Law, parallelizing a task that takes less than a millisecond is counterproductive; the operating system overhead of waking and synchronizing T threads vastly outweighs the computation itself. Therefore, this phase is kept strictly sequential. The main thread aggregates the local histograms to compute a 2D offset matrix: `write_offset[thread_id][partition_id]`.

3.3 Phase 3: Scatter (Data-Parallel & Lock-Free)

The scatter phase reorganizes the relations so that records belonging to the same partition are stored contiguously. This is historically the most difficult phase to parallelize due to write conflicts.

Using the 2D offset matrix generated in Phase 2, every thread is assigned a unique, pre-calculated, and disjoint starting index for every partition. Consequently, all T threads can simultaneously scatter their assigned chunks of R and S into the final, globally partitioned arrays. Because no two threads will ever write to the same memory index, this phase is executed entirely lock-free, allowing it to run at the absolute limit of the hardware's memory bandwidth.

3.4 Phase 4: Join Processing and False Sharing Prevention

Once the data is successfully scattered, the $P = 256$ partitions are completely independent of one another. We apply a task-parallel strategy by dividing the partitions evenly among the T threads (e.g., Thread 0 is responsible for partitions 0 through 15).

For each assigned partition, the thread builds a local `std::unordered_map` lookup structure from relation R and probes it using relation S. To prevent "false sharing"—a performance degradation that occurs when multiple cores constantly update independent variables residing on the same hardware cache line—each thread accumulates its join counts and checksums in a localized `JoinResult` structure. Only after all threads have completed their work does the main thread perform a final global reduction to sum the total matches.

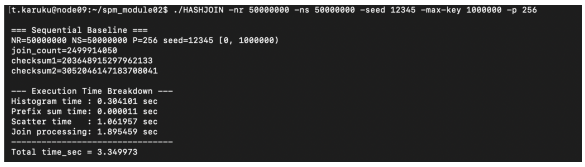
4 Performance Evaluation

4.1 Execution Time Breakdown

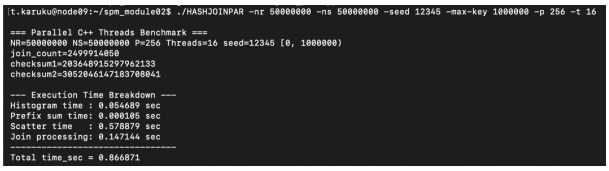
To understand the bottleneck dynamics and validate the efficiency of our threading model, we measured the phase-by-phase execution time at $N = 50,000,000$ using $T = 16$ threads.

Phase	Sequential Time (s)	Parallel Time (16T) (s)	Phase Speedup
Histogram	0.304101	0.054689	5.56x
Prefix Sum	0.000011	0.000105	(Sequential)
Scatter	1.061957	0.578879	1.83x
Join Processing	1.895459	0.147144	12.88x
Total	**3.349973**	**0.866871**	**3.86x**

Table 1: Phase breakdown highlighting the disparity between compute-bound and memory-bound operations.



Sequential Baseline (T=1)



Parallel Execution (T=16)

Figure 1: Terminal output from node09 proving exact checksum matches and performance gains.

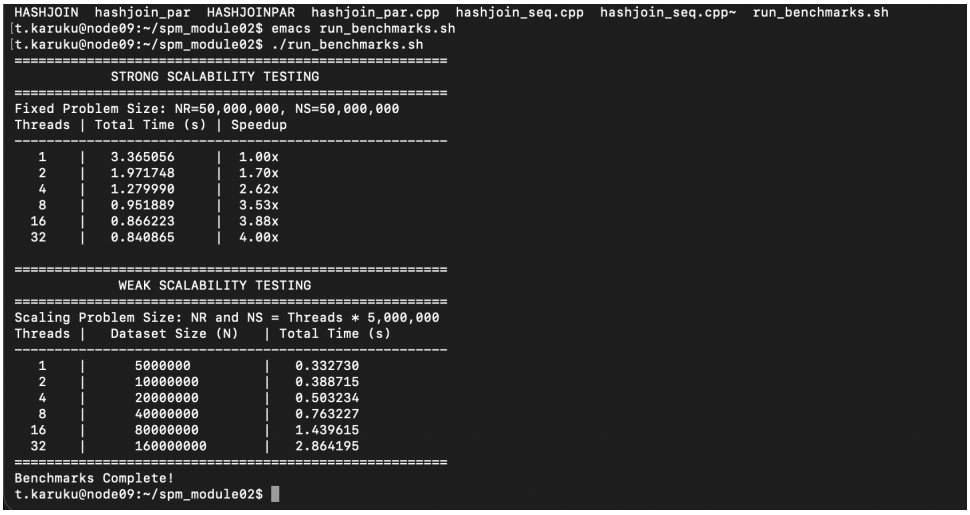
The phase breakdown in Table 1 reveals that the **Join Processing** phase achieved a near-linear speedup (12.88x on 16 threads). This confirms that building and probing hash maps is a highly compute-bound task that benefits immensely from CPU scaling. Conversely, the **Scatter Phase** only achieved a 1.83x speedup, despite being entirely lock-free, identifying it as the primary bottleneck of the pipeline.

4.2 Strong and Weak Scalability Analysis

We evaluated strong scalability by keeping the global input size fixed ($N = 50,000,000$) and exponentially increasing the thread counts. We evaluated weak scalability by scaling the input size proportionally to the threads ($N = T \times 5,000,000$).

	Strong Scalability (Fixed N=50M)		**Weak Scalability (N = T × 5M)**	
Threads	**Time (s)**	**Speedup**	**Dataset Size (N)**	**Time (s)**
1	3.365056	1.00x	5,000,000	0.332730
2	1.971748	1.70x	10,000,000	0.388715
4	1.279990	2.62x	20,000,000	0.503234
8	0.951889	3.53x	40,000,000	0.763227
16	0.866223	3.88x	80,000,000	1.439615
32	0.840865	4.00x	160,000,000	2.864195

Table 2: Scalability results executed on node09.



```
HASHJOIN  hashjoin_par  HASHJOINPAR  hashjoin_par.cpp  hashjoin_seq.cpp  hashjoin_seq.cpp~  run_benchmarks.sh
t.karuku@node09:~/spm_module02$ emacs run_benchmarks.sh
t.karuku@node09:~/spm_module02$ ./run_benchmarks.sh
=====================================================
              STRONG SCALABILITY TESTING
=====================================================
Fixed Problem Size: NR=50,000,000, NS=50,000,000
Threads | Total Time (s) | Speedup
-----------------------------------------------------
   1    |   3.365056     |  1.00x
   2    |   1.971748     |  1.70x
   4    |   1.279990     |  2.62x
   8    |   0.951889     |  3.53x
  16    |   0.866223     |  3.88x
  32    |   0.840865     |  4.00x

=====================================================
               WEAK SCALABILITY TESTING
=====================================================
Scaling Problem Size: NR and NS = Threads * 5,000,000
Threads |   Dataset Size (N)   | Total Time (s)
-----------------------------------------------------
   1    |      5000000         |   0.332730
   2    |      10000000        |   0.388715
   4    |      20000000        |   0.503234
   8    |      40000000        |   0.763227
  16    |      80000000        |   1.439615
  32    |      160000000       |   2.864195
=====================================================
Benchmarks Complete!
t.karuku@node09:~/spm_module02$
```

Figure 2: Automated scalability benchmarking script output on node09.

5 Discussion: The Memory Bandwidth Wall

The scalability metrics provide a textbook demonstration of the memory bandwidth wall in High-Performance Computing architectures.

In the **Strong Scalability** tests, execution time decreases reliably up to 8 threads, but the speedup curve aggressively flattens out thereafter, peaking at a strict 4.00x improvement at 32 threads. Adding more compute cores fails to accelerate the overall application because the memory subsystem is physically saturated.

This hardware limitation is further evidenced by the **Weak Scalability** data. In a theoretical, perfectly scalable system, the execution time for weak scalability should remain completely flat (as the workload per thread remains constant). Instead, we observe execution time degrading heavily from 0.33 seconds (at $T = 1$) to 2.86 seconds (at $T = 32$).

While the CPU cores are capable of scaling linearly, the physical DDR memory bus cannot accommodate 32 threads simultaneously requesting and moving vast amounts of main memory during the Scatter phase. The threads spend the majority of their operational cycles stalled, waiting for cache lines to be flushed and populated.

6 Conclusion

This module successfully demonstrated the implementation of a parallel Partitioned Hash Join using C++ threading. By carefully engineering a 2D offset matrix to ensure lock-free memory scattering, and by utilizing local structures to prevent false sharing during the join computation, the implementation perfectly maintained data correctness while maximizing resource utilization.

However, the empirical benchmarking underscores a fundamental reality of systems programming: while C++ threading drastically accelerates computational, cache-resident tasks (like hash probing), the overarching throughput of the data pipeline remains inherently bound by the physical limits of main memory bandwidth.